

Variable et conditionnelle Python



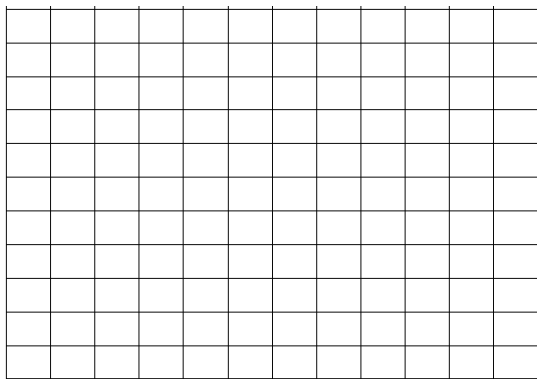
Exercice 1 : Serpent-art

Pour les trois situations ci-dessous, réaliser le dessin correspondant en démarrant le dessin à partir du coin en bas à gauche en allant vers la droite. Puis compléter chaque tableau en indiquant pour chaque ligne du programme la valeur de trait et d'angle.

```

1      trait = 6
2      angle = 90
3      avancer(trait)
4      tourner_gauche(angle)
5      avancer(trait)
6      trait = 3
7      tourner_droite(angle)
8      avancer(trait)

```

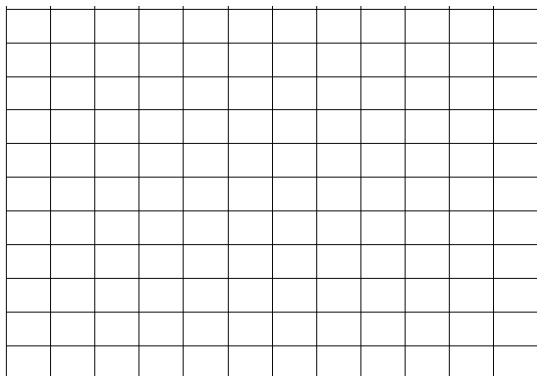


	trait	angle
ligne 1		
ligne 2		
ligne 3		
ligne 4		
ligne 5		
ligne 6		
ligne 7		
ligne 8		

```

1      grand_trait = 6
2      angle = 90
3      avancer(grand_trait)
4      tourner_gauche(angle)
5      avancer(grand_trait)
6      petit_trait = 2
7      tourner_droite(angle)
8      avancer(petit_trait)
9      tourner_droite(angle)
10     grand_trait = grand_trait - petit_trait
11     avancer(grand_trait)

```

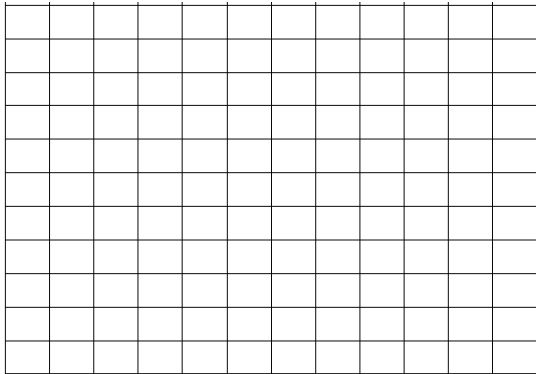


	trait	angle
ligne 1		
ligne 2		
ligne 3		
ligne 4		
ligne 5		
ligne 6		
ligne 7		
ligne 8		
ligne 9		
ligne 10		
ligne 11		

```

1      petit_trait = 1
2      grand_trait = 3
3      angle = 90
4      avancer(petit_trait)
5      tourner_gauche(angle)
6      avancer(grand_trait)
7      tourner_droite(angle)
8      petit_trait = grand_trait
9      grand_trait = petit_trait * 2
10     avancer(grand_trait)

```



	trait	angle
ligne 1		
ligne 2		
ligne 3		
ligne 4		
ligne 5		
ligne 6		
ligne 7		
ligne 8		
ligne 9		
ligne 10		



Cours 1 : Variable et affectation

Un programme est une suite d'**instructions** permettant de manipuler des données. Ces données sont stockées par la machine et qui pourront, selon le programme, être utilisées ou modifiées. Elles portent le nom de **variable**, car bien souvent il s'agit de valeurs qui évolueront en fonction du déroulement du programme.

Une **variable** correspond à une **affectation** (= association) nom, valeur, c'est ce qui s'appelle la **déclaration**.

```

1      # nom variable = valeur
2      compte_banque = 500
3      loyer = 270
4      compte_banque = compte_banque - loyer

```

Dans le programme ci-dessus, la variable **compte_banque** est déclarée et la valeur 500 lui est affectée. Ces deux informations sont stockées en mémoire par la machine et à chaque fois que la variable sera appelée, la machine fera automatiquement le lien entre le nom et la valeur.

Quand le loyer est prélevé du compte bancaire, la machine transforme automatiquement la ligne :
compte_banque = compte_banque - loyer par **compte_banque = 500 - 270**.



Cours 2 : Traçabilité de programme

Réaliser la **trace** dans un programme signifie suivre l'évolution des valeurs de celui-ci. Cela permet d'analyser ce que fait un programme pendant son exécution et pouvoir identifier d'éventuelles erreurs qui s'y seraient glissées.

On représente généralement la trace par un tableau. Chaque ligne représente une ligne du programme, tandis que les colonnes représentent les valeurs des différentes variables du programme.



Exercice 2 : 1 mouton, 2 moutons, 3 moutons,...

Compléter le programme suivant afin qu'il y ait 10 moutons dans la variable champ à la fin de l'exécution en utilisant uniquement l'opérateur + et sans utiliser de nombre.

```

1      # Definition des variables
2      mouton = 1
3      champ = 0
4
5      # Programme principal
6      mouton = .....
7      mouton = .....
8      mouton = .....
9      champ = .....
```



Exercice 3 : Trace-hison, dis-trace

A partir de la trace suivante, compléter le code sans utiliser de nombres.

```

1      # Definition des variables
2      point_permis = 6
3      annee_conduite = 2
4      infraction = 4
5      stage = 2
6
7      # Programme principal
8      ...
9      ...
10     ...
11     ...
12     ...
13     ...
```

instruction ligne	point_permis	annee_conduite	infraction	stage
2	6	/	/	/
3	6	2	/	/
4	6	2	4	/
5	6	2	4	2
8	8	2	4	2
9	4	2	4	2
10	4	2	8	2
11	-12	2	8	2
12	-10	2	8	2
13	-8	2	8	2



Exercice 4 : Le monde à l'envers

Inverser le contenu des deux variables ci-dessous, sans utiliser de nombre.

```

1      grand = 3
2      petit = 7
```



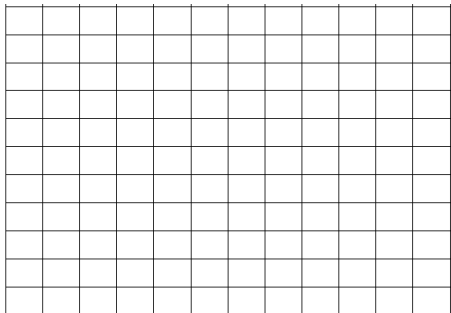
Exercice 5 : P-if-elart

Pour les deux situations ci-dessous, réaliser le dessin correspondant en démarrant le dessin à partir du coin en bas à gauche en allant vers la droite. Puis compléter chaque tableau en indiquant pour chaque ligne du programme la valeur de trait et de angle.

```

1      grand_trait = 5
2      petit_trait = 3
3      angle = 90
4      avancer(grand_trait)
5      tourner_gauche(angle)
6      avancer(petit_trait)
7      if petit_trait > 4:
8          avancer(grand_trait)

```

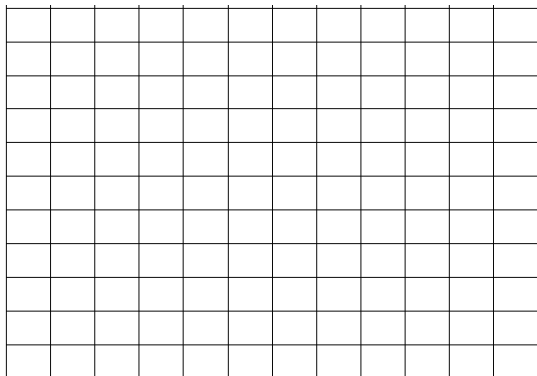


	trait	angle
ligne 1		
ligne 2		
ligne 3		
ligne 4		
ligne 5		
ligne 6		
ligne 7		
ligne 8		

```

1      grand_trait = 5
2      petit_trait = 3
3      angle = 90
4      if ((petit_trait + grand_trait) / 2) > 4:
5          trait = 2
6      else:
7          trait = 1
8      avancer(trait)
9      tourner_gauche(angle)
10     trait = trait * 2
11     avancer(trait)
12     tourner_droite(angle)
13     trait = trait * 3
14     avancer(trait)

```



	trait	angle
ligne 1		
ligne 2		
ligne 3		
ligne 4		
ligne 5		
ligne 6		
ligne 7		
ligne 8		
ligne 9		
ligne 10		
ligne 11		
ligne 12		
ligne 13		
ligne 14		



Cours 3 : Conditionnelle

Dans un programme Python, il est parfois souhaitable que certaines actions soient réalisées uniquement dans un cas précis. Il s'agit donc en quelques sortes d'un aiguillage, si la condition est respectée alors le code contenu est exécuté, ce « test d'entrée » est appelé **conditionnelle**. Dans le cas contraire, l'exécution reprend directement après la conditionnelle comme si elle n'existait pas. Le mot-clef Python permettant de réaliser des conditionnelles est le **if**.

Lorsque l'on souhaite changer le code en fonction du résultat d'une conditionnelle, on peut en plus utiliser les mots-clefs **elif** (abréviation de else if) et **else**. Ces instructions veulent dire « sinon si » et « sinon », et sont à utiliser comme en français.

Si ma note est plus grande que 16, j'ai les félicitations, sinon si ma note est plus grande que 14, j'ai les compliments, sinon si ma note est plus grande que 12, j'ai les encouragements, sinon je n'ai rien.

```

1      if (condition):
2          bloc d'instructions 1
3
4      elif (condition):
5          bloc d'instructions 2
6
7      elif (condition):
8          bloc d'instructions 2
9      ...
10
11     else:
12         bloc d'instructions n

```



Exercice 6 : Can'if

Qu'affiche le programme suivant ?

```

1      chien = "Rottweiler"
2
3      if chien == "caniche":
4          print("C'est un petit chien.")
5
6      else:
7          print("C'est peut-être un petit chien.")

```

Qu'afficherait le programme si la variable chien valait "Caniche" ?



Exercice 7 : Ver-if-ication

Lorsque l'on souhaite réaliser un achat, une vérification est demandée à la banque afin de s'assurer que notre compte bancaire contienne suffisamment d'argent. Compléter le code suivant afin de respecter cette condition.

```

1      valeur_achat = 154
2      compte = 200
3
4      # on veut pouvoir déduire une somme d'argent à condition que le compte soit solvable
5      if ..... > ..... :

```

```

6         compte = ..... - .....
7         # sinon on affiche que le compte n'est pas solvable
8     else :
9         print("Paiement refusé")

```



Exercice 8 : Je suis met'if

On considère le programme suivant :

```

1     couleur_salon = "bleu"
2
3     if couleur_salon == "bleu" :
4         couleur_salon = "jaune"
5
6     if couleur_salon == "jaune" :
7         couleur_salon = "bleu"

```

Quel sera la valeur de la variable couleur_salon à la fin de l'exécution du programme ?



Exercice 9 : if-Saint-Laurent

A partir du texte ci-dessous, on essaye de réaliser un code python pour calculer le total à payer d'un client. Compléter le code Python.

Dans un magasin de parfum, si on possède un compte fidélité, une réduction de 50% est appliquée. De plus, pour les fêtes, une réduction supplémentaire est proposée. Si 1 article est acheté, 5€ de réduction, si 2 articles sont achetés, 15€ de réduction, et pour 3 ou plus 50€ de réduction.

```

1
2     # j'ai un compte de fidélité
3     ...
4     # j'ai acheté 2 articles
5     ...
6     # mon total à payer est de 185
7     ...
8
9     # si je possède un compte de fidélité
10    ...
11    # si j'ai acheté 1 article
12    ...
13    # le total à payer diminue de 5
14    ...
15    # sinon si j'ai acheté 2 articles
16    ...
17    # le total à payer diminue de 15
18    ...
19    # sinon
20    ...
21    # le total à payer diminue de 50
22    ...
23    # le total est divisé par 2
24    ...
25    # sinon
26    ...

```

```

27         # si j'ai acheté 1 article
28         ...
29         # le total à payer diminue de 5
30         ...
31         # sinon si j'ai acheté 2 articles
32         ...
33         # le total à payer diminue de 15
34         ...
35         # sinon
36         ...
37         # le total à payer diminue de 50
38         ...

```



Exercice 10 : Nutrit-if

Une entreprise souhaite automatiser sa classification de produits en ligne à l'aide d'un programme. L'entreprise propose trois produits : les fleurs, les fruits et les légumes. Si le produit n'est pas comestible, alors l'étiquette du produit devra indiquer qu'il s'agit d'une fleur. Sinon, il va falloir distinguer les fruits des légumes. Si le produit est sucré alors l'étiquette du produit devra indiquer qu'il s'agit d'un fruit, sinon l'étiquette du produit devra indiquer qu'il s'agit d'un légume.

Écrire le programme correspondant.



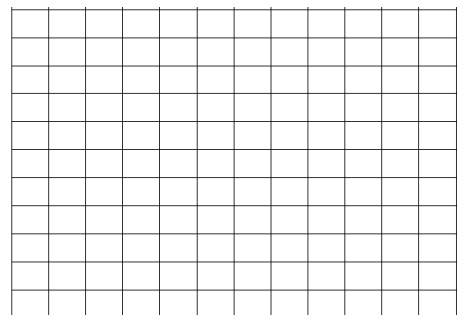
Exercice 11 : Fest-if

Le code ci-dessous permet de réaliser un dessin. Réaliser ce dessin à côté du programme.

```

1         a = 5
2         b = 3
3         c = 90
4         g(a)
5         f(c)
6         g(b)
7         if b > 4:
8             g(a)

```



Cours 4 : Nommage et commentaire

Il est important pour un programmeur de respecter plusieurs règles lorsqu'il programme. Il doit toujours faciliter la compréhension et la relecture du programme, pour lui-même mais aussi si son programme est repris par autrui : c'est-à-dire donner des noms clairs et simples lorsque l'on manipule des données, ajouter des commentaires pertinents, etc.



Cours 5 : Nommage et PEP8

Il est essentiel d'utiliser des noms de variables explicites lorsque l'on programme. Les noms de variables doivent respecter plusieurs règles :

- utiliser des caractères alphanumériques, des underscores (_) ou bien des chiffres, mais pas de caractères spéciaux (accent, ponctuation, espace, etc.) ;
- commencer par une lettre ;
- ne pas utiliser des mots-clés de Python (if, def, try, ...).

En effet, un bon nommage permet une meilleure lisibilité de votre code car les noms de variable peuvent permettre de conserver le contexte. De plus, il est souvent conseillé de donner un nom court et pertinent à une variable. Un nom trop long peut lui aussi nuire à la compréhension.

Il existe, au sein de la communauté Python, un ensemble de règles (nommées **PEP8**) regroupant les bonnes pratiques de nommage afin de faciliter et d'homogénéiser le partage de code. En voici un extrait :

- ne pas dépasser les 79 caractères sur une même ligne ;
- les noms de variables s'écrivent en lettres minuscules ;
- si le nom de variable contient plusieurs mots, il est nécessaire de les séparer par un _ (exemple : fleur_bleue) ;
- les opérateurs (+, -, =, /, *, ==, !=, >=, not, in, and, or, etc.) doivent être entourés d'espace (exemple : compte = compte + vente) ;
- il n'y a pas d'espace entre les crochets, les parenthèses et les accolades (exemple : print(diviseur(3))) ;
- il n'y a pas d'espace avant les parenthèses associées à une fonction ou les crochets d'une liste (exemple : liste_eleves[2]) ;
- il est nécessaire de mettre un espace après les caractères : et , mais pas avant (exemple : liste_notes = [5, 18, 20]) ;
- la déclaration de variables se trouve en haut des fichiers de programmation ou des fonctions, afin de les distinguer du reste du programme.



Exercice 12 : Select'if

Parmi les variables suivantes, entourer celles qui respectent la PEP8.

t	compte_bancaire	ma_super_belle_variable	resultat	points
azerty	sec_nombre	infraction	else	a_payer
voiture	6	premier_nombre	intermédiaire	temp
AnneConduite	malvariable	notes	4	nbr_vies
console	2nombre	mon adresse	nombre_Pommes	somme

Objectifs :

Vous êtes désormais capables :

- de créer et manipuler des variables ;
- de réaliser la trace d'un programme ;
- d'écrire des conditionnelles ;
- de commenter un programme ;
- de citer des règles de PEP8.

Fonction Python

Réflexion : Fonction-naire

Analyser les codes suivants, que remarquez-vous ?

1	prendre_crayon("red")	prendre_crayon("green")	prendre_crayon("blue")
2	tracer_ligne(8)	tracer_ligne(8)	tracer_ligne(4)
3	tourner_gauche()	tourner_gauche()	tourner_gauche()
4	tracer_ligne(8)	tracer_ligne(8)	tracer_ligne(4)
5	tourner_gauche()	tourner_gauche()	tourner_gauche()
6	tracer_ligne(8)	tracer_ligne(8)	tracer_ligne(4)
7	tourner_gauche()	tourner_gauche()	tourner_gauche()
8	tracer_ligne(8)	tracer_ligne(8)	tracer_ligne(4)



Cours 6 : Principe d'une fonction

Une **fonction** Python va correspondre à une suite d'instructions, qui nous sera utile à plusieurs reprises dans un programme. Cela permet donc d'éviter la répétition de code dans un programme. Les instructions effectuées au sein de la fonction peuvent tenir compte de **paramètres** qui seront passés à la fonction. Les paramètres servent à modifier les valeurs d'un même comportement. Par exemple, quand on souhaite effectuer l'action se lever, on peut le faire lentement ou rapidement. Si se lever était une fonction, elle posséderait un paramètre vitesse indiquant à quelle intensité cette action doit s'effectuer.

Fonction correspondant à l'encadré de réflexion :

```

1  def dessiner_carre(couleur, taille):
2      '''
3          dessine un carré coloré à l'aide de couleur et de dimension taille
4      '''
5      prendre_crayon(couleur)
6      tracer_ligne(taille)
7      tourner_gauche()
8      tracer_ligne(taille)
9      tourner_gauche()
10     tracer_ligne(taille)
11     tourner_gauche()
12     tracer_ligne(taille)
13
14     # appel de la fonction
15     dessiner_carre("red", 8)
16     dessiner_carre("green", 8)
17     dessiner_carre("blue", 4)

```

Une fonction se déclare à l'aide du mot-clef **def** suivi du nom de la fonction, des paramètres éventuels entre parenthèses et du caractère **:**. Exemple :

```

1  def nom_fonction(param1, param2, ...):
2      instructions

```

Il est important d'**indenter** (décaler) le code au sein d'une fonction. Quand on souhaite **appeler** une fonction (c'est-à-dire l'utiliser), on indique simplement le nom de la fonction et on lui donne des **arguments** (des valeurs pour chaque paramètre). A chaque fois que l'interpréteur Python lira un nom de fonction, il exécutera automatiquement le code indenté situé sous la fonction.



Exercice 13 : Per-fonction

Reprendre le code de l'exercice 7. Créer une fonction du nom de votre choix, qui prendra en paramètres un montant de compte en banque, et une valeur d'achat. La fonction affichera (print) si l'achat peut être fait ou non.



Exercice 14 : Fonction lominaire

Pour chacune des questions suivantes, créer la fonction correspondante, puis l'appeler avec les paramètres de votre choix.

1. Écrire une fonction rectangle, qui prend en paramètres une largeur, une hauteur, et une couleur. Cette fonction devra dessiner le rectangle avec les bonnes valeurs.

2. Écrire une fonction triangle_equilateral, qui prend en paramètres une taille. Cette fonction devra dessiner le triangle (rappel : un triangle équilatéral a 3 côtés de même longueur, et des angles de 60°).

3. Écrire une fonction dessine, qui prend en paramètres un nom de figure, une première taille, une seconde taille et une couleur. La fonction dessine doit appeler la bonne fonction de dessin, avec les bons arguments.



Activité 1 : Ki-local-orie

Écrire le code suivant sous Python.

```
1      # declaration de variables
2      gateau = 10
3      personnes = 2
4
5      # declaration des fonctions
6      def partage(valeur, nombre):
7          return valeur // nombre
8
9      # programme principal
10     resultat = partage(gateau, personnes)
11     print(gateau)
12     print(resultat)
13     print(valeur)
```

1. Quelles sont les valeurs affichées pour les variables gateau, resultat, valeur ?

2. Supprimer le mot-clef return. Quelles sont les valeurs affichées pour les variables gateau, resultat, valeur ?

3. D'après vous, à quoi sert le mot-clef return ?



Cours 7 : Variable locale et return

Tout ce qui existe dans une fonction n'existe pas en dehors, on parle de **variable locale**.

Pour pouvoir partager un résultat calculé par une fonction, la fonction doit **retourner** ce résultat. Le mot-clef Python permettant cela est **return**. Ce mot-clef n'existe que pour les fonctions, on ne peut pas le retrouver en dehors d'une fonction. Pour ré-utiliser le résultat renvoyé par une fonction, il faut ensuite le stocker dans une variable.

Attention : le mot-clef **return** arrête l'exécution de la fonction. Tout ce qui se trouvera ensuite ne sera pas exécuté.



Activité 2 : Grandeur nature

Écrire une fonction `plus_grand` prenant en paramètres deux nombres. Cette fonction renverra le plus grand des deux nombres.



Cours 8 : Type

En Python, les variables possèdent un type en fonction de leurs contenus. Contrairement à d'autres langages, il n'est pas nécessaire de **spécifier** (c'est-à-dire indiquer) le type d'une variable, ceci sera fait de façon **dynamique** (automatiquement en fonction du contenu de la variable) par Python. En fonction du type attribué à une donnée, elle ne sera pas stockée sur le même espace mémoire par la machine.

Il existe quatre types principaux en Python :

- les entiers : type `int` ;
- les décimaux : type `float` ;
- les chaînes de caractères : type `str` ;
- les booléens (True ou False) : type `bool`.

Il est possible de connaître le type d'une variable à l'aide de la fonction native `type()`.



Exercice 15 : Qui suis-je ?

Quel est le type des données suivantes ?

	508	"Titre"	3.14	"8"	True	"False"	98.	.798
int								
float								
str								
bool								



Cours 9 : Signature de fonctions

Quand on écrit une fonction, la première étape consiste à réfléchir à sa **signature** (aussi appelée spécification de fonction). Cela consiste à réfléchir aux paramètres d'entrée (leur nombre et leur type) ainsi qu'à ce qui sera renvoyer par la fonction (type des données renvoyées par la fonction). Ces informations sont ensuite notées et partagées dans la **documentation** afin que toute personne ayant besoin d'utiliser la fonction puisse le faire correctement.

En Python, ces informations sont indiquées sous le nom de la fonction ; c'est la **docstring**. On y retrouve le rôle, les informations sur les données d'entrée et de sortie. Reprenons l'exemple de la fonction donnée dans l'encadré de cours 6, sa signature complète est la suivante :

```

1      def dessiner_carre(couleur, taille):
2          '''
3              dessine un carré coloré à l'aide de couleur et de dimension taille
4              :param couleur: la couleur du carré
5              :type couleur: str
6              :param taille: la taille du carré
7              :type taille: int
8              :return: X
9              :rtype: X
10         '''

```



Exercice 16 : Ba-signe-se

Pour chaque question, écrire la fonction correspondante, en écrivant bien la spécification complète.

1. Écrire une fonction `addition`, qui prend en paramètres deux entiers, et renvoie leur somme.

2. Écrire une fonction `plus_petit`, qui prend en paramètres trois nombres décimaux, et renvoie le plus petit.



Activité 3 : For-midable

Écrire et tester le code suivant sous Python.

```
1     jour = 1
2     print(jour)
3
4     for _ in range(30):
5         jour = jour + 1
6         print(jour)
```

1. Qu'est-ce qui s'affiche ?

2. Modifier le nombre de la ligne `for _ in range()`. Que se passe-t-il ?

3. Désindenter la ligne 6 pour la mettre au même niveau que la ligne 4. Que se passe-t-il ?

4. D'après vous, à quoi sert une boucle `for` ?



Cours 10 : Print

Pour faire un retour à l'utilisateur, ou pour aider le développeur à suivre la bonne exécution de son programme et corriger des erreurs, on peut demander à l'ordinateur d'afficher des choses. Pour se faire, on utilise la fonction `print`, qui permet notamment d'afficher les valeurs de ce qui se trouve en paramètres.

Attention, il est important de distinguer l'utilisation du `return`, de l'utilisation d'un `print` (une fonction native de Python). Un `print` n'a pour seul but que l'affichage, c'est-à-dire écrire dans le terminal, tandis que le `return` permet de renvoyer une valeur pour être stockée / réutilisée dans la suite du programme.

Ainsi, sur le code ci-dessous, on a défini une fonction qui permet de renvoyer le résultat de la multiplication de deux nombres, on l'appelle une fois et on stocke le résultat obtenu dans une variable. Puis pour vérifier ou connaître, le résultat obtenu, on affiche celui-ci à l'aide de la fonction `print`.

```
1      def multiplication(prem_nb, sec_nb):
2          '''
3              multiplie les deux nombres
4              :param prem_nb: une première valeur
5              :type prem_nb: int
6              :param sec_nb: une seconde valeur
7              :type sec_nb: int
8              :return: res
9              :rtype: int
10         '''
11         res = prem_nb * sec_nb
12         return res
13
14     resultat = multiplication(5, 5)
15     print(resultat)
```



Exercice 17 : Suivre à la trace

On considère la fonction suivante :

```
1      def maj_compte(compte, prix):
2          if compte > 0:
3              compte - prix
4          return compte
5
6      compte = 13
7      achat = 47
8      compte = maj_compte(compte, prix)
9      if (compte < 0):
10         for _ in range(5):
11             compte = compte + 10
```



Exercice 18 : A-somme-z moi

Écrire une fonction réalisant la somme de deux valeurs à l'aide d'une boucle for.

Objectifs :

Vous êtes désormais capables :

- d'écrire et d'appeler des fonctions ;
- d'écrire la spécification de la fonction ;
- de récupérer le résultat renvoyé par une fonction ;
- d'identifier les types d'une donnée ;
- différencier print et return.

Boucles Python



Activité 4 : 1 10

Écrire et tester le code suivant sous Python

```
1         for chiffre in range(10):  
2             print(chiffre)
```

1. A quoi correspond l'affichage du programme ?

2. Quelle est la première valeur affichée ?

3. Quelle est la dernière valeur affichée ?

4. Qu'en déduisez-vous sur l'utilisation de la boucle for ?



Exercice 19 : May the for be with you

On considère le code Python ci-dessous :

```
1         somme = 0  
2         for chiffre in range(10):  
3             somme = somme + chiffre
```

1. Réaliser la trace de la variable somme.

2. Transformer ce programme en fonction prenant en paramètres le nombre de tours de boucle qui sera réalisé et renvoyant la somme calculée.



Cours 11 : Boucle for

En python, il est possible de répéter plusieurs fois un même bloc d'instruction grâce à une boucle bornée : la **boucle pour**. On parle de boucle bornée car cette boucle va itérer (= se répéter) un nombre défini de fois.

En Python, pour écrire une boucle bornée, on utilise le mot-clef **for**. Quand on définit une boucle à l'aide du **for**, on peut l'écrire de 3 façons :

Le premier cas est celui vu précédemment dans les exercices. La boucle va s'exécuter en commençant à 0 jusqu'à atteindre la valeur stop en incrémentant la valeur de 1 à chaque tour de boucles.

```
for nom in range(stop):
    serie d'instructions
```

Si nous souhaitons que la boucle démarre à une autre valeur que 0, il est possible d'ajouter une valeur de départ. La boucle va s'exécuter en commençant à la valeur start jusqu'à atteindre la valeur stop en incrémentant la valeur de 1 à chaque tour de boucles.

```
for nom in range(start, stop):
    serie d'instructions
```

Enfin, il est possible d'ajouter un pas (ou saut) dans la fonction range. A chaque tour de boucle, l'incrément ne se fera pas de 1 mais du pas qui a été déclaré. Cela peut-être utile si l'on veut par exemple aller de deux en deux, de trois en trois, etc.

```
for nom in range(start, stop, pas):
    serie d'instructions
```



Exercice 20 : Pas à pas

On considère le code Python ci-dessous :

```
1     somme = 0
2     for chiffre in range(2, 10):
3         somme = chiffre + somme
```

1. Réaliser la trace des variables somme et chiffre.

2. Transformer ce programme en fonction prenant en paramètres la valeur de départ de la boucle, la valeur de fin de la boucle, et renvoyant la somme calculée.

On considère le code Python ci-dessous :

```
1     somme = 0
2     for chiffre in range(15, 30, 4):
3         somme = chiffre + somme
```

3. Réaliser la trace des variables somme et chiffre.

4. Transformer ce programme en fonction prenant en paramètres la valeur de départ de la boucle, la valeur de fin de la boucle, le pas et renvoyant la somme calculée.



Exercice 21 : For-tnite

La ville de Londres souhaite compter le nombre de touristes visitant Big Ben à l'aide des tour bus. Elle souhaite pour cela écrire un programme permettant d'automatiser le calcul.

1. Écrire une fonction `compteur_bus` qui prend en paramètres un nombre de bus et un nombre de visiteurs moyens présents dans ces bus. Cette fonction renverra le nombre total de visiteurs et devra obligatoirement utiliser une boucle `for`.

2. Écrire une fonction `compteur_mois` qui prend en paramètres le nombre de jours du mois et un nombre de visiteurs journaliers. Cette fonction renvoie le nombre de visiteurs total durant le mois, en prenant en compte que tous les sept jours (les dimanches), l'affluence est moitié moins forte que le reste de la semaine.

3. Écrire l'instruction permettant d'appeler la fonction `compteur_mois` sur le mois de février en considérant qu'en moyenne il y a 124 visiteurs par jour.



Activité 5 : Ha-while

Écrire et tester le code suivant sous Python

```
1     nb = 49
2     racine = 0
3     for chiffre in range(nb):
4         if chiffre * chiffre == nb:
5             racine = chiffre
6         print(chiffre)
7     print(racine)
```

1. Que permet de faire ce code ? Que pouvez-vous dire dessus ?

```

1      nb = 49
2      racine = 0
3      chiffre = 0
4      while chiffre * chiffre != nb:
5          print(chiffre)
6          chiffre = chiffre + 1
7      racine = chiffre
8      print(racine)

```

2. Que permet de faire ce second code ? Quelle différence majeure d'exécution peut-on remarquer ?

3. Que se passe-t-il si on cherche la racine de 50 ?



Cours 12 : Boucle while

Il existe un deuxième type de boucle, nommé **boucle conditionnelle** (ou boucle tant que). A la différence d'une boucle bornée, une boucle conditionnelle va s'exécuter **tant qu'une condition est respectée**. A chaque tour de boucles, la condition est testée afin de savoir si on effectue un nouveau tour de boucles (la condition est toujours vraie) ou si on arrête la boucle (la condition est devenue fausse).

En Python, le mot-clef pour ce type de boucle est **while**.

```

1      while (conditions):
2          instructions
3          mise à jour des conditions

```

Contrairement à la boucle pour, il est nécessaire de manipuler avec précaution la boucle tant que. En effet, cette boucle est dépendante d'une condition pour s'arrêter. Si celle-ci n'est pas correctement traitée, la boucle peut ne jamais s'arrêter.

Pour éviter la situation de boucle infinie bloquant le programme, il est nécessaire de respecter trois étapes lorsque l'on souhaite utiliser la boucle **while** :

- **initialisation** : déclarer la variable qui va évoluer AVANT la boucle ;
- **condition d'arrêt** : définir la condition qui permettra d'arrêter la boucle ;
- **mise à jour** : modifier, dans la boucle, la variable afin de se rapprocher de la condition d'arrêt.



Exercice 22 : Man vs while

Transformer le code suivant, de façon à utiliser une boucle While.

```

1      code = 15
2      tentative = 0

```

```

3     for entier in range(100):
4         tentative = tentative + 1
5         if (code == tentative):
6             print("Code trouvé")

```



Exercice 23 : L'histoire sans fin

Parmi les 4 codes suivants, entourer ceux qui ne se terminent pas, puis justifier pourquoi.

```

nombre_cheveux = 0
implant = -1
while nombre_cheveux <= 0:
    nombre_cheveux = nombre_cheveux + implant

```

```

code = 7485
nombre = 0
while nombre != code:
    nombre = nombre + 1

```

```

compte_bancaire = 50
while compte_bancaire != 100:
    compte_bancaire = compte_bancaire + 3

```

```

nb_vie_joueur = 3
while nb_vie_joueur > 0:
    nb_vie_joueur = nb_vie_joueur - 1

```



Exercice 24 : Trace ton chemin

Faire la trace du programme suivant

```

1     pv_joueur = 20
2     pv_boss = 1000
3     dgt_joueur = 67
4     dgt_boss = 2
5     nb_tour = 0
6     chargement_attaque = 0
7
8     while pv_joueur > 0 and pv_boss > 0:
9         pv_boss = pv_boss - dgt_joueur
10        if chargement_attaque == 2:
11            pv_joueur = pv_joueur - dgt_boss
12            chargement_attaque = 0
13        else:
14            chargement_attaque = chargement_attaque + 1
15            nb_tour = nb_tour + 1
16        print(nb_tour)

```

Objectifs :

Vous êtes désormais capable de :

- utiliser les boucles pour :
- construire une boucle tant que ;
- différencier les cas d'utilisation d'une boucle pour d'une boucle tant que ;
- d'identifier des situations provoquant des boucles infinies.

Mise en pratique



Cours 14 : Algorithme

Un **algorithme** est une suite d'instructions et d'opérations à suivre afin de résoudre un problème. Exemple, on souhaite traverser la route de manière sûre :

1. se rendre au passage piéton
2. attendre que le feu piéton passe au vert
3. traverser

Quand un algorithme est écrit dans un langage naturel, on parle de **pseudo-code**. En informatique, il y a toujours une phase de réflexion papier avant de passer à l'**implémentation** sur machine. Cette phase de réflexion s'effectue en général en **pseudo-code**, c'est-à-dire dans un langage non machine. Il est ensuite facile pour un développeur de convertir le **pseudo-code** dans le langage machine qu'il souhaite (dans notre cas Python).



Activité 6 : Plus ou moins

Le jeu du plus ou moins est un jeu où le but est de deviner une valeur décidée par un premier joueur en début de jeu. A chaque tour, le second joueur va proposer une valeur et le premier va alors donner une information permettant d'aider le second joueur ; il va indiquer si la valeur proposée est inférieure, supérieure ou égale à la valeur décidée.

1. Proposer les étapes d'un algorithme correspondant à ce jeu.

2. Rappeler les deux types de boucles étudiées en cours et leur méthode de fonctionnement.

3. D'après vous, quel type de boucles est le plus adapté pour ce jeu ? Justifier.

4. Écrire la boucle de jeu.

On considère le code suivant :

```
1     import random
2
3     nombre = random.randint(100)
4     print(nombre)
```

5. Tester le code précédent. Quelle valeur avez-vous obtenu ?

6. Comparer le résultat avec vos voisins. En déduire ce que permet de faire ce code. Indice : vous pouvez également le relancer pour observer ce qu'il se passe.

7. Créer une fonction permettant de générer un nombre aléatoire. Cette fonction prendra en paramètre un nombre qui correspondra à la borne maximale et retournera une valeur générée aléatoirement entre 0 et la borne maximale.

8. Créer une fonction compare permettant de comparer deux nombres passés en paramètres. Cette fonction affichera si le premier nombre est plus grand, plus petit ou égal au second nombre. Aide : on considère que le premier nombre est le nombre à trouver et le second le nombre proposé.

On considère le code suivant :

```
1 phrase = input("Ecrire puis appuyer sur Entrée : ")
2 print(phrase)
```

9. Tester le code précédent puis saisir un texte au clavier et terminer en appuyant sur la touche Entrée. D'après vous, que permet de faire ce code ?

10. Implémenter le jeu en utilisant les fonctions précédemment créées et en réutilisant votre boucle de jeu.

11. Tester votre programme. Que se passe-t-il ?

12. A l'aide du message d'erreur, identifier la (ou les) ligne(s) de code qui pose(nt) problème.

13. D'après vous, et à l'aide de vos connaissances, que peut signifier cette erreur ?

14. Corriger l'erreur, puis tester à nouveau votre programme.

15. (Pour les plus rapides) Modifier votre code afin de permettre à un autre joueur de choisir le nombre à deviner au lieu de le tirer aléatoirement.

16. (Pour les plus rapides) Rajouter une fonction dans votre programme permettant de vérifier que la prochaine valeur entrée par le second joueur respecte le résultat de la fonction compare. C'est-à-dire : si le nombre proposé est plus grand que la valeur à trouver, alors le joueur doit proposer un nombre plus petit que ce qu'il a proposé précédemment, sinon le jeu lui redemande de rentrer une valeur.

17. (Pour les plus rapides) Rajouter un système d'essai. Le second joueur a désormais 10 essais pour deviner la valeur.



Cours 15 : Bibliothèque Python

Une **bibliothèque** en informatique est un regroupement de fonctions mises à disposition des développeurs afin de faciliter leur travail. Cela permet d'éviter d'écrire des fonctions déjà existantes et ainsi économiser du temps.

Chaque bibliothèque officielle de Python dispose d'une documentation en ligne, souvent disponible sur le site de documentation officielle de Python : docs.python.org.



Cours 16 : Construction de programme

Un programme est décomposé en plusieurs sections permettant de simplifier la lecture du code :

```
1      # la section d'import
```

```
2
3     # la section de déclaration
4
5     # la section du programme principal
```

Comme son nom l'indique, la section d'import consiste à déclarer toutes les bibliothèques Python qui seront utilisées dans notre programme. Elle se situe en haut du programme.

A la suite, on trouve la section de déclaration. C'est ici que l'on retrouve toutes les fonctions créées pour notre programme. Chaque fonction possède sa docstring permettant de comprendre son rôle et son fonctionnement.

Enfin, la fin du fichier est consacré au programme principal. C'est dans celui-ci que les fonctions déclarées sont appelées (= utilisées). Quand on exécute un fichier Python c'est cette partie du programme qui va être exécutée.



Cours 17 : Input, typage et erreur

La fonction `input` en Python permet d'interagir avec un utilisateur en récupérant des données entrées au clavier. Elle est utile lorsque l'on a besoin d'informations sur celui-ci. Toutes données entrées avec la fonction `input` est renvoyée dans le type `str` (chaîne de caractères) pour pouvoir être ensuite traitées par le reste du programme. Il est donc important par la suite de vérifier la validité des données entrées par l'utilisateur et de prendre en compte les conversions de typage éventuelles pouvant avoir lieu par la suite.

Des problèmes de typage entraînent (bien souvent) des déclenchements d'erreurs en Python (`TypeError`), mais ce n'est pas toujours le cas, ce qui peut provoquer des erreurs plus tard dans le programme ou des comportements inattendus. Il est donc nécessaire d'être vigilant aux types des données manipulées.

Remarque : Informations supplémentaires - Tour de boucles

Programme 1

```
tour = 0
score = 50
while (tour < 30):
    score = score + 1
    tour = tour + 1
```

Programme 2

```
tour = 30
score = 50
for an in range(tour):
    score = score + 1
    tour = tour + 1
```

Observez ces deux programmes. Dans le premier programme, la variable `tour` est comparée après chaque terminaison du bloc d'instructions qui suit. Tant que la variable aura une valeur inférieure à 30, la boucle continuera de s'exécuter. La variable impliquée dans la condition est mise à jour à chaque tour de boucle. Ce dernier point est **capital** afin d'éviter les boucles dites infinies, c'est-à-dire qui ne s'arrête jamais. Dans le second programme, l'ordinateur va lire une seule fois la ligne `for` et remplacer automatiquement la variable `tour` par sa valeur initiale (grâce à un itérateur). La boucle s'effectuera donc 30 fois malgré le changement de valeurs de la variable `tour`.

Objectifs :

Vous êtes désormais capables :

- d'expliquer la notion d'algorithmes ;
- de proposer un algorithme correspondant à une situation donnée ;
- de traduire un algorithme en langage Python ;
- d'identifier une erreur de typage ;
- de transmettre des valeurs entre différentes fonctions du programme.